

Introduction to Deep Learning

Deep Learning from Scratch with NumPy
Diabetes Prediction Using a 3-Layer Neural Network

Lecturer: Dinh Que Tran
Intelligent System Development

No TensorFlow No PyTorch No Keras
We build a small Deep Learning model ourselves using NumPy.

1 Learning Objectives

After completing this tutorial, students should be able to:

1. Explain Deep Learning as a composition of parameterized functions.
2. Distinguish traditional machine learning from representation learning.
3. Explain what a neural network layer does.
4. Implement a 3-layer neural network using only NumPy.
5. Understand forward propagation.
6. Understand ReLU and sigmoid activation functions.
7. Understand binary cross-entropy loss.
8. Understand backpropagation conceptually and computationally.
9. Implement gradient descent manually.
10. Train a neural network for diabetes prediction.
11. Evaluate the trained model using accuracy, precision, recall, and F1.
12. Understand what Deep Learning frameworks such as PyTorch automate.

2 The Central Idea of Deep Learning

One of the most important ideas in this course is:

Key Concept

Deep Learning = Composition of learned functions

Instead of learning only one direct mapping from input to output, Deep Learning constructs several transformations and learns the parameters of these transformations.

Suppose that the input is represented by

$$X.$$

A traditional model might learn approximately

$$\hat{y} = f(X).$$

A Deep Learning model instead constructs

$$X \longrightarrow f_1(X) \longrightarrow f_2(f_1(X)) \longrightarrow f_3(f_2(f_1(X))).$$

Therefore,

$$\hat{y} = f_3(f_2(f_1(X)))$$

or, using function composition notation,

$$\hat{y} = (f_3 \circ f_2 \circ f_1)(X)$$

This idea will guide the entire tutorial.

3 Traditional Machine Learning vs. Deep Learning

Consider the diabetes prediction problem.

The input contains patient information such as:

- Number of pregnancies
- Glucose concentration
- Blood pressure
- Skin thickness
- Insulin
- Body mass index
- Diabetes pedigree function
- Age

A traditional ML approach often follows:

$$\text{Data} \rightarrow \text{Feature Engineering} \rightarrow \text{ML Model} \rightarrow \text{Prediction}$$

The human designer may decide which transformations or features are useful.

Deep Learning changes the philosophy:

$$\text{Data} \rightarrow \text{Learned Representations} \rightarrow \text{Prediction}$$

The model itself learns intermediate representations.

Key Concept

Traditional ML:

Human-designed features $\rightarrow$ learn prediction function

Deep Learning:

learn representations $\rightarrow$ learn prediction function

4 Our First Deep Learning Architecture

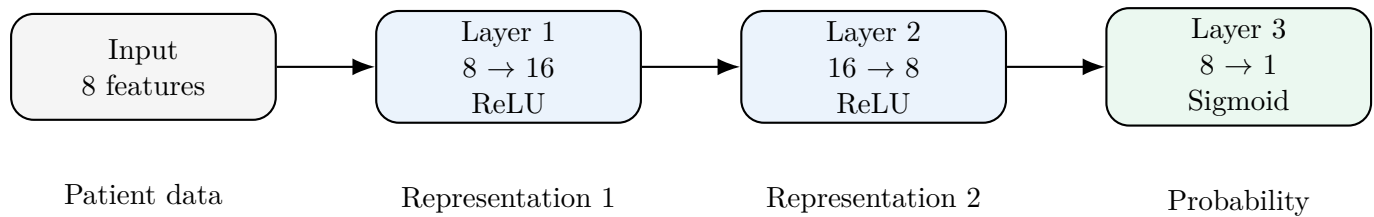
We will construct the following neural network:

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

where:

- 8 input features
- 16 neurons in Layer 1
- 8 neurons in Layer 2
- 1 output neuron

There are therefore three trainable layers.



The important conceptual structure is:

$$X \xrightarrow{f_1} H_1 \xrightarrow{f_2} H_2 \xrightarrow{f_3} \hat{y}$$

5 The Input Data

We use the Pima Indians Diabetes Dataset.

Each patient is described by eight numerical input variables.

Index	Feature	Meaning
0	Pregnancies	Number of pregnancies
1	Glucose	Plasma glucose concentration
2	BloodPressure	Blood pressure
3	SkinThickness	Skin fold thickness
4	Insulin	Serum insulin
5	BMI	Body mass index
6	DiabetesPedigreeFunction	Diabetes pedigree score
7	Age	Age

The target is:

$$y \in \{0, 1\}.$$

We interpret:

$$y = 0 \quad \Rightarrow \quad \text{No diabetes}$$

and

$$y = 1 \quad \Rightarrow \quad \text{Diabetes.}$$

Thus the problem is **binary classification**.

6 Layer 1: The First Learned Function

The first layer performs a linear transformation:

$$Z_1 = XW_1 + b_1.$$

Then we apply an activation function:

$$H_1 = \text{ReLU}(Z_1).$$

Therefore:

$$f_1(X) = \text{ReLU}(XW_1 + b_1)$$

The dimensions are:

$$X \in \mathbb{R}^{N \times 8}$$

$$W_1 \in \mathbb{R}^{8 \times 16}$$

$$b_1 \in \mathbb{R}^{1 \times 16}$$

and therefore

$$H_1 \in \mathbb{R}^{N \times 16}.$$

The first layer transforms eight original features into sixteen learned representations.

6.1 What Does One Neuron Do?

Consider one neuron.

It receives:

$$x_1, x_2, \dots, x_8.$$

It computes:

$$z = w_1x_1 + w_2x_2 + \dots + w_8x_8 + b.$$

Then:

$$h = \text{ReLU}(z).$$

So a neuron learns how to combine the input features.

Sixteen neurons learn sixteen different combinations.

For Students

The weights are not manually designed by us.
 They are learned from data.
 This is the beginning of **representation learning**.

7 The ReLU Function

The Rectified Linear Unit is

$$\text{ReLU}(x) = \max(0, x)$$

Therefore,

$$\text{ReLU}(x) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

For example:

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(0) = 0$$

$$\text{ReLU}(4) = 4.$$

Its derivative is approximately

$$\text{ReLU}'(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0. \end{cases}$$

ReLU introduces nonlinearity into the network.

Important

Without nonlinear activation functions, stacking several linear layers would still produce one overall linear transformation.
 Therefore, nonlinear activations are essential for learning complex functions.

8 Layer 2: Learning from Learned Representations

Layer 2 receives H_1 rather than the original patient features.

It computes:

$$Z_2 = H_1 W_2 + b_2$$

and

$$H_2 = \text{ReLU}(Z_2).$$

Therefore:

$$f_2(H_1) = \text{ReLU}(H_1 W_2 + b_2)$$

with

$$W_2 \in \mathbb{R}^{16 \times 8}.$$

The transformation is:

$$16 \rightarrow 8.$$

This gives an important interpretation:

Layer 1 learns representations from data.

Then:

Layer 2 learns representations from representations.

This is one of the fundamental ideas behind Deep Learning.

9 Layer 3: Producing the Prediction

The final layer computes:

$$Z_3 = H_2 W_3 + b_3.$$

Then we apply sigmoid:

$$\hat{y} = \sigma(Z_3).$$

The sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

and

$$0 < \sigma(z) < 1.$$

Therefore the output can be interpreted as a probability-like score.
For example:

$$\hat{y} = 0.91$$

means that the model assigns a high probability to class 1.

The final decision can be:

$$\hat{y} \geq 0.5 \Rightarrow \text{Diabetes}$$

and

$$\hat{y} < 0.5 \Rightarrow \text{No Diabetes}.$$

10 The Complete Mathematical Model

We now combine all three layers.

Layer 1:

$$H_1 = \text{ReLU}(XW_1 + b_1).$$

Layer 2:

$$H_2 = \text{ReLU}(H_1W_2 + b_2).$$

Layer 3:

$$\hat{y} = \sigma(H_2W_3 + b_3).$$

Therefore:

$$\hat{y} = \sigma(\text{ReLU}(\text{ReLU}(XW_1 + b_1)W_2 + b_2)W_3 + b_3)$$

This is our Deep Learning model.

Key Concept

The entire neural network can be understood as:

$$\hat{y} = f_3(f_2(f_1(X)))$$

This is **function composition**.

11 Forward Propagation

The process of calculating a prediction from input data is called **forward propagation**.

The computation proceeds from left to right:

$$X \rightarrow Z_1 \rightarrow H_1 \rightarrow Z_2 \rightarrow H_2 \rightarrow Z_3 \rightarrow \hat{y}.$$

More explicitly:

$$Z_1 = XW_1 + b_1$$

$$H_1 = \text{ReLU}(Z_1)$$

$$Z_2 = H_1W_2 + b_2$$

$$H_2 = \text{ReLU}(Z_2)$$

$$Z_3 = H_2W_3 + b_3$$

$$\hat{y} = \sigma(Z_3).$$

12 Why Do We Need Learning?

Initially, the weights are random.

Therefore the network will produce poor predictions.

For example:

$$y = 1, \quad \hat{y} = 0.23.$$

The model is wrong.

We need a way to measure how wrong it is.

This is the role of the **loss function**.

13 Binary Cross-Entropy Loss

For binary classification, we use Binary Cross-Entropy:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Consider:

$$y = 1.$$

If the model predicts

$$\hat{y} = 0.99,$$

the loss is small.

If the model predicts

$$\hat{y} = 0.01,$$

the loss is very large.

Therefore, the loss tells the network:

How bad are my current predictions?

14 Learning = Reducing the Loss

The overall learning problem can be represented as:

Find weights that minimize the loss

That is,

$$\theta^* = \arg \min_{\theta} L(\theta)$$

where

$$\theta = \{W_1, b_1, W_2, b_2, W_3, b_3\}.$$

The network therefore learns its parameters from data.

15 Backpropagation

Once the loss has been calculated, we need to determine how each parameter contributed to the error.

This is the purpose of **backpropagation**.

For example:

$$\frac{\partial L}{\partial W_3}$$

tells us how the loss changes when W_3 changes.

Similarly,

$$\frac{\partial L}{\partial W_2}$$

and

$$\frac{\partial L}{\partial W_1}$$

tell us how the loss depends on earlier layers.

The chain rule allows the gradients to propagate backward through the network.

16 The Chain Rule

Suppose:

$$L = L(H_2)$$

and

$$H_2 = H_2(H_1)$$

and

$$H_1 = H_1(W_1).$$

Then:

$$\boxed{\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial H_2} \frac{\partial H_2}{\partial H_1} \frac{\partial H_1}{\partial W_1}}$$

This is the mathematical foundation of backpropagation.

Important

Backpropagation is not a mysterious Deep Learning operation.

It is the systematic application of the chain rule to a composition of functions.

17 Gradient Descent

Once we know the gradient, we update the weights.

For a generic parameter W :

$$\boxed{W \leftarrow W - \eta \frac{\partial L}{\partial W}}$$

where η is the learning rate.
For example:

$$W_3 \leftarrow W_3 - \eta \frac{\partial L}{\partial W_3}.$$

This is gradient descent.
The complete learning cycle is therefore:

Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update

and repeated many times:

Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update $\rightarrow \dots$

18 Implementing Deep Learning with NumPy

We now implement everything ourselves.

We will use:

- NumPy
- Matrix multiplication
- ReLU
- Sigmoid
- Binary cross-entropy
- Backpropagation
- Gradient descent

We will **not** use:

- TensorFlow
- PyTorch
- Keras
- scikit-learn

19 Step 1: Import NumPy

```
1 import numpy as np
```

NumPy provides arrays and efficient matrix operations.
For example:

```
1 X @ W
```

implements matrix multiplication.

20 Step 2: Load the Dataset

The dataset should be saved as:

diabetes.csv

The code is:

```
1 data = np.loadtxt(  
2     "diabetes.csv",  
3     delimiter="," ,  
4     skiprows=1  
5 )  
6  
7 X = data[:, :8]  
8 y = data[:, 8].reshape(-1, 1)  
9  
10 print("X shape:", X.shape)  
11 print("y shape:", y.shape)
```

Expected shapes are approximately:

$X : (768, 8)$

and

$y : (768, 1)$.

21 Step 3: Split the Data

We randomly divide the data into training and testing sets.

```
1 np.random.seed(42)  
2  
3 indices = np.random.permutation(len(X))  
4  
5 split = int(0.8 * len(X))  
6  
7 train_idx = indices[:split]  
8 test_idx = indices[split:]  
9  
10 X_train = X[train_idx]  
11 y_train = y[train_idx]  
12  
13 X_test = X[test_idx]  
14 y_test = y[test_idx]  
15  
16 print("Training samples:", len(X_train))  
17 print("Testing samples :", len(X_test))
```

The training data is used to learn the parameters.

The testing data is used to evaluate generalization.

22 Step 4: Normalize the Input

Different features may have very different scales.

For example, glucose and age are on different numerical scales.

We standardize each feature:

$$x' = \frac{x - \mu}{\sigma}$$

where μ and σ are calculated from the training data.

```

1 mean = X_train.mean(axis=0)
2 std = X_train.std(axis=0) + 1e-8
3
4 X_train = (X_train - mean) / std
5 X_test = (X_test - mean) / std

```

Notice that the test set uses the training mean and standard deviation.
This prevents information from the test set leaking into training.

23 Step 5: Implement ReLU

```

1 def relu(x):
2     return np.maximum(0, x)

```

The derivative is:

```

1 def relu_derivative(x):
2     return (x > 0).astype(float)

```

Mathematically:

$$\text{ReLU}'(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0. \end{cases}$$

24 Step 6: Implement Sigmoid

```

1 def sigmoid(x):
2
3     x = np.clip(x, -50, 50)
4
5     return 1.0 / (1.0 + np.exp(-x))

```

The derivative can be written as:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

In the implementation we can pass the already computed sigmoid value.

```

1 def sigmoid_derivative(a):
2     return a * (1.0 - a)

```

25 Step 7: Initialize the Network

Our architecture is:

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 1.$$

Therefore:

$$W_1 \in \mathbb{R}^{8 \times 16}$$

$$W_2 \in \mathbb{R}^{16 \times 8}$$

$$W_3 \in \mathbb{R}^{8 \times 1}.$$

The code is:

```

1 np.random.seed(42)
2
3 W1 = np.random.randn(8, 16) * np.sqrt(2.0 / 8)
4 b1 = np.zeros((1, 16))
5
6 W2 = np.random.randn(16, 8) * np.sqrt(2.0 / 16)
7 b2 = np.zeros((1, 8))
8
9 W3 = np.random.randn(8, 1) * np.sqrt(2.0 / 8)
10 b3 = np.zeros((1, 1))

```

The weights start from small random values.

The biases start at zero.

26 Understanding the Matrix Dimensions

Suppose we have N patients.

Then:

$$X \in \mathbb{R}^{N \times 8}.$$

For Layer 1:

$$XW_1$$

has dimensions:

$$(N \times 8)(8 \times 16) = N \times 16.$$

Therefore:

$$H_1 \in \mathbb{R}^{N \times 16}.$$

Layer 2:

$$(N \times 16)(16 \times 8) = N \times 8.$$

Therefore:

$$H_2 \in \mathbb{R}^{N \times 8}.$$

Layer 3:

$$(N \times 8)(8 \times 1) = N \times 1.$$

Therefore:

$$\hat{y} \in \mathbb{R}^{N \times 1}.$$

For Students

Understanding these dimensions is extremely important.
A neural network is largely a sequence of matrix transformations plus nonlinear functions.

27 Step 8: Forward Propagation Code

```

1 def forward(X):
2
3     # Layer 1
4     z1 = X @ W1 + b1
5     h1 = relu(z1)
6
7     # Layer 2
8     z2 = h1 @ W2 + b2
9     h2 = relu(z2)
10
11    # Layer 3
12    z3 = h2 @ W3 + b3
13    y_hat = sigmoid(z3)
14
15    cache = {
16        "X": X,
17        "z1": z1,
18        "h1": h1,
19        "z2": z2,
20        "h2": h2,
21        "z3": z3,
22        "y_hat": y_hat
23    }
24
25    return y_hat, cache

```

This code corresponds directly to the mathematics.

$$Z_1 = XW_1 + b_1$$

$$H_1 = \text{ReLU}(Z_1)$$

$$Z_2 = H_1W_2 + b_2$$

$$H_2 = \text{ReLU}(Z_2)$$

$$Z_3 = H_2W_3 + b_3$$

$$\hat{y} = \sigma(Z_3).$$

28 Why Do We Save the Intermediate Values?

Notice that the forward function stores:

$$Z_1, H_1, Z_2, H_2, Z_3, \hat{y}.$$

These values are needed during backpropagation.

For example, the derivative of ReLU requires knowing whether:

$$Z_1 > 0.$$

Therefore, the forward pass creates information that the backward pass will use.

This is exactly the basic idea behind computational graphs used by modern Deep Learning frameworks.

29 Step 9: Binary Cross-Entropy

```

1 def binary_cross_entropy(y, y_hat):
2
3     eps = 1e-8
4
5     y_hat = np.clip(
6         y_hat,
7         eps,
8         1 - eps
9     )
10
11     loss = -np.mean(
12         y * np.log(y_hat)
13         +
14         (1 - y) * np.log(1 - y_hat)
15     )
16
17     return loss

```

The small value ϵ prevents numerical problems such as:

$$\log(0).$$

30 Step 10: Backpropagation

We now implement the backward pass.

```

1 def backward(y, cache):
2
3     X = cache["X"]
4     z1 = cache["z1"]
5     h1 = cache["h1"]
6     z2 = cache["z2"]
7     h2 = cache["h2"]
8     yhat = cache["y_hat"]
9
10    n = len(X)
11
12    # -----
13    # Layer 3
14    # -----
15
16    dz3 = (yhat - y) / n
17
18    dW3 = h2.T @ dz3
19
20    db3 = np.sum(

```

```

21     dz3,
22     axis=0,
23     keepdims=True
24 )
25
26 # -----
27 # Layer 2
28 # -----
29
30 dh2 = dz3 @ W3.T
31
32 dz2 = dh2 * relu_derivative(z2)
33
34 dW2 = h1.T @ dz2
35
36 db2 = np.sum(
37     dz2,
38     axis=0,
39     keepdims=True
40 )
41
42 # -----
43 # Layer 1
44 # -----
45
46 dh1 = dz2 @ W2.T
47
48 dz1 = dh1 * relu_derivative(z1)
49
50 dW1 = X.T @ dz1
51
52 db1 = np.sum(
53     dz1,
54     axis=0,
55     keepdims=True
56 )
57
58 gradients = {
59     "dW1": dW1,
60     "db1": db1,
61     "dW2": dW2,
62     "db2": db2,
63     "dW3": dW3,
64     "db3": db3
65 }
66
67 return gradients

```

31 Understanding Backpropagation Step by Step

For the final layer, using sigmoid together with binary cross-entropy gives the particularly convenient result:

$$\frac{\partial L}{\partial Z_3} = \hat{y} - y$$

Therefore:

$$dZ_3 = \hat{y} - y.$$

Then:

$$dW_3 = H_2^T dZ_3$$

and

$$db_3 = \sum dZ_3.$$

Next, the gradient propagates backward:

$$dH_2 = dZ_3 W_3^T.$$

Because

$$H_2 = \text{ReLU}(Z_2),$$

we have

$$dZ_2 = dH_2 \odot \text{ReLU}'(Z_2).$$

Then:

$$dW_2 = H_1^T dZ_2.$$

Finally:

$$dH_1 = dZ_2 W_2^T$$

and

$$dZ_1 = dH_1 \odot \text{ReLU}'(Z_1).$$

Therefore:

$$dW_1 = X^T dZ_1.$$

This is backpropagation.

32 The Backward Flow

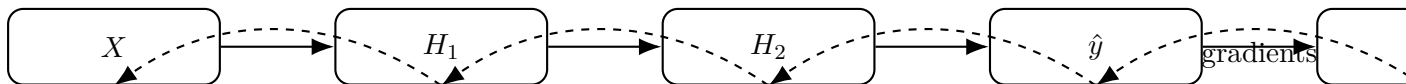
The forward direction is:

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y} \rightarrow L.$$

The backward direction is:

$$L \rightarrow W_3 \rightarrow W_2 \rightarrow W_1.$$

Thus:



33 Step 11: Gradient Descent

We choose a learning rate:

$$\eta = 0.01.$$

Then update all trainable parameters:

```

1 W1 -= learning_rate * gradients["dW1"]
2 b1 -= learning_rate * gradients["db1"]
3
4 W2 -= learning_rate * gradients["dW2"]
5 b2 -= learning_rate * gradients["db2"]
6
7 W3 -= learning_rate * gradients["dW3"]
8 b3 -= learning_rate * gradients["db3"]

```

Mathematically:

$$W_1 \leftarrow W_1 - \eta \frac{\partial L}{\partial W_1}$$

$$W_2 \leftarrow W_2 - \eta \frac{\partial L}{\partial W_2}$$

$$W_3 \leftarrow W_3 - \eta \frac{\partial L}{\partial W_3}.$$

The same idea applies to biases.

34 Step 12: Training Loop

The complete training procedure is:

```

1 learning_rate = 0.01
2 epochs = 1000
3
4 for epoch in range(epochs):
5
6     # Forward
7     y_hat, cache = forward(X_train)
8
9     # Loss
10    loss = binary_cross_entropy(
11        y_train,
12        y_hat
13    )
14
15    # Backward
16    gradients = backward(
17        y_train,
18        cache
19    )
20
21    # Gradient descent
22    W1 -= learning_rate * gradients["dW1"]
23    b1 -= learning_rate * gradients["db1"]
24
25    W2 -= learning_rate * gradients["dW2"]
26    b2 -= learning_rate * gradients["db2"]

```

```

27
28 W3 -= learning_rate * gradients["dW3"]
29 b3 -= learning_rate * gradients["db3"]
30
31 if epoch % 100 == 0:
32     print(
33         f"Epoch {epoch:4d} | "
34         f"Loss = {loss:.4f}"
35     )

```

This loop is the heart of learning.

35 What Happens During One Epoch?

One training iteration can be understood as:

$$\boxed{X_{\text{train}}}$$

goes forward:

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y}.$$

Then:

$$\hat{y} \rightarrow L.$$

Then:

$$L \rightarrow \nabla W_3, \nabla W_2, \nabla W_1.$$

Finally:

$$W \rightarrow W - \eta \nabla W.$$

Therefore:

$$\boxed{\text{One training step} = \text{Forward} + \text{Loss} + \text{Backward} + \text{Update}}$$

36 Step 13: Prediction

After training, we calculate predictions on unseen test data.

```

1 y_prob, _ = forward(X_test)
2
3 y_pred = (
4     y_prob >= 0.5
5 ).astype(int)

```

There are two outputs:

$$y_{\text{prob}}$$

and

$$y_{\text{pred}}.$$

For example:

Probability	Prediction	Meaning
0.12	0	No diabetes
0.31	0	No diabetes
0.51	1	Diabetes
0.87	1	Diabetes
0.96	1	Diabetes

37 Step 14: Accuracy

Accuracy is:

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

The code is:

```

1 accuracy = np.mean(
2     y_pred == y_test
3 )
4
5 print("Accuracy:", accuracy)
```

38 Step 15: Confusion Matrix

For binary classification we define:

TP = True Positive

TN = True Negative

FP = False Positive

FN = False Negative.

The code is:

```

1 TP = np.sum(
2     (y_pred == 1) &
3     (y_test == 1)
4 )
5
6 TN = np.sum(
7     (y_pred == 0) &
8     (y_test == 0)
9 )
10
11 FP = np.sum(
12     (y_pred == 1) &
13     (y_test == 0)
14 )
15
16 FN = np.sum(
17     (y_pred == 0) &
18     (y_test == 1)
19 )
```

39 Precision, Recall and F1

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

The implementation is:

```

1 precision = TP / (TP + FP + 1e-8)
2
3 recall = TP / (TP + FN + 1e-8)
4
5 f1 = (
6     2 * precision * recall
7     /
8     (precision + recall + 1e-8)
9 )
10
11 print("Precision:", precision)
12 print("Recall :", recall)
13 print("F1 :", f1)

```

40 Complete Program

The following program puts everything together.

```

1 # =====
2 # DEEP LEARNING FROM SCRATCH
3 # Diabetes Prediction
4 #
5 # NO TensorFlow
6 # NO PyTorch
7 # NO Keras
8 # NO Scikit-learn
9 #
10 # Only NumPy is used for the neural network.
11 # =====
12
13 import numpy as np
14
15
16 # =====
17 # 1. LOAD DATA
18 # =====
19
20 data = np.loadtxt(
21     "diabetes.csv",
22     delimiter=",",

```

```
23     skiprows=1
24 )
25
26 X = data[:, :8]
27 y = data[:, 8].reshape(-1, 1)
28
29 print("X shape:", X.shape)
30 print("y shape:", y.shape)
31
32
33 # =====
34 # 2. TRAIN / TEST SPLIT
35 # =====
36
37 np.random.seed(42)
38
39 indices = np.random.permutation(len(X))
40
41 split = int(0.8 * len(X))
42
43 train_idx = indices[:split]
44 test_idx = indices[split:]
45
46 X_train = X[train_idx]
47 y_train = y[train_idx]
48
49 X_test = X[test_idx]
50 y_test = y[test_idx]
51
52 print("Training samples:", len(X_train))
53 print("Testing samples :", len(X_test))
54
55
56 # =====
57 # 3. NORMALIZATION
58 # =====
59
60 mean = X_train.mean(axis=0)
61 std = X_train.std(axis=0) + 1e-8
62
63 X_train = (X_train - mean) / std
64 X_test = (X_test - mean) / std
65
66
67 # =====
68 # 4. ACTIVATION FUNCTIONS
69 # =====
70
71 def relu(x):
72     return np.maximum(0, x)
73
74
75 def relu_derivative(x):
76     return (x > 0).astype(float)
77
78
79 def sigmoid(x):
80
81     x = np.clip(x, -50, 50)
```

```
83     return 1.0 / (1.0 + np.exp(-x))
84
85
86 # =====
87 # 5. INITIALIZE NETWORK
88 # =====
89
90 np.random.seed(42)
91
92 # Layer 1: 8 -> 16
93
94 W1 = (
95     np.random.randn(8, 16)
96     * np.sqrt(2.0 / 8)
97 )
98
99 b1 = np.zeros((1, 16))
100
101
102 # Layer 2: 16 -> 8
103
104 W2 = (
105     np.random.randn(16, 8)
106     * np.sqrt(2.0 / 16)
107 )
108
109 b2 = np.zeros((1, 8))
110
111
112 # Layer 3: 8 -> 1
113
114 W3 = (
115     np.random.randn(8, 1)
116     * np.sqrt(2.0 / 8)
117 )
118
119 b3 = np.zeros((1, 1))
120
121
122 # =====
123 # 6. FORWARD PROPAGATION
124 # =====
125
126 def forward(X):
127
128     # Layer 1
129
130     z1 = X @ W1 + b1
131
132     h1 = relu(z1)
133
134
135     # Layer 2
136
137     z2 = h1 @ W2 + b2
138
139     h2 = relu(z2)
140
141
142     # Layer 3
```

```

143
144     z3 = h2 @ W3 + b3
145
146     y_hat = sigmoid(z3)
147
148
149     cache = {
150         "X": X,
151         "z1": z1,
152         "h1": h1,
153         "z2": z2,
154         "h2": h2,
155         "z3": z3,
156         "y_hat": y_hat
157     }
158
159     return y_hat, cache
160
161
162 # =====
163 # 7. LOSS
164 # =====
165
166 def binary_cross_entropy(y, y_hat):
167
168     eps = 1e-8
169
170     y_hat = np.clip(
171         y_hat,
172         eps,
173         1 - eps
174     )
175
176     loss = -np.mean(
177         y * np.log(y_hat)
178         +
179         (1 - y)
180         * np.log(1 - y_hat)
181     )
182
183     return loss
184
185
186 # =====
187 # 8. BACKPROPAGATION
188 # =====
189
190 def backward(y, cache):
191
192     X = cache["X"]
193
194     z1 = cache["z1"]
195     h1 = cache["h1"]
196
197     z2 = cache["z2"]
198     h2 = cache["h2"]
199
200     yhat = cache["y_hat"]
201
202     n = len(X)

```

```
203
204
205 # -----
206 # Layer 3
207 # -----
208
209 dz3 = (yhat - y) / n
210
211 dW3 = h2.T @ dz3
212
213 db3 = np.sum(
214     dz3,
215     axis=0,
216     keepdims=True
217 )
218
219
220 # -----
221 # Layer 2
222 # -----
223
224 dh2 = dz3 @ W3.T
225
226 dz2 = (
227     dh2
228     * relu_derivative(z2)
229 )
230
231 dW2 = h1.T @ dz2
232
233 db2 = np.sum(
234     dz2,
235     axis=0,
236     keepdims=True
237 )
238
239
240 # -----
241 # Layer 1
242 # -----
243
244 dh1 = dz2 @ W2.T
245
246 dz1 = (
247     dh1
248     * relu_derivative(z1)
249 )
250
251 dW1 = X.T @ dz1
252
253 db1 = np.sum(
254     dz1,
255     axis=0,
256     keepdims=True
257 )
258
259
260 gradients = {
261     "dW1": dW1,
262     "db1": db1,
```

```
263     "dW2": dW2,
264     "db2": db2,
265     "dW3": dW3,
266     "db3": db3
267 }
268
269 return gradients
270
271
272 # =====
273 # 9. TRAINING
274 # =====
275
276 learning_rate = 0.01
277 epochs = 1000
278
279 for epoch in range(epochs):
280
281     # Forward
282
283     y_hat, cache = forward(
284         X_train
285     )
286
287
288     # Loss
289
290     loss = binary_cross_entropy(
291         y_train,
292         y_hat
293     )
294
295
296     # Backward
297
298     gradients = backward(
299         y_train,
300         cache
301     )
302
303
304     # Update
305
306     W1 -= (
307         learning_rate
308         * gradients["dW1"]
309     )
310
311     b1 -= (
312         learning_rate
313         * gradients["db1"]
314     )
315
316     W2 -= (
317         learning_rate
318         * gradients["dW2"]
319     )
320
321     b2 -= (
322         learning_rate
```

```

323     * gradients["db2"]
324 )
325
326 W3 -= (
327     learning_rate
328     * gradients["dw3"]
329 )
330
331 b3 -= (
332     learning_rate
333     * gradients["db3"]
334 )
335
336
337 if epoch % 100 == 0:
338
339     print(
340         f"Epoch {epoch:4d} | "
341         f"Loss = {loss:.4f}"
342     )
343
344
345 # =====
346 # 10. TEST
347 # =====
348
349 y_prob, _ = forward(
350     X_test
351 )
352
353 y_pred = (
354     y_prob >= 0.5
355 ).astype(int)
356
357
358 # =====
359 # 11. ACCURACY
360 # =====
361
362 accuracy = np.mean(
363     y_pred == y_test
364 )
365
366 print()
367 print("=====")
368 print("TEST RESULTS")
369 print("=====")
370
371 print(
372     "Accuracy:",
373     accuracy
374 )
375
376
377 # =====
378 # 12. CONFUSION MATRIX
379 # =====
380
381 TP = np.sum(
382     (y_pred == 1)

```

```

383     &
384     (y_test == 1)
385 )
386
387 TN = np.sum(
388     (y_pred == 0)
389     &
390     (y_test == 0)
391 )
392
393 FP = np.sum(
394     (y_pred == 1)
395     &
396     (y_test == 0)
397 )
398
399 FN = np.sum(
400     (y_pred == 0)
401     &
402     (y_test == 1)
403 )
404
405
406 print()
407 print("Confusion Matrix")
408 print("-----")
409
410 print("TP:", TP)
411 print("TN:", TN)
412 print("FP:", FP)
413 print("FN:", FN)
414
415
416 # =====
417 # 13. PRECISION / RECALL / F1
418 # =====
419
420 precision = (
421     TP
422     /
423     (TP + FP + 1e-8)
424 )
425
426 recall = (
427     TP
428     /
429     (TP + FN + 1e-8)
430 )
431
432 f1 = (
433     2
434     * precision
435     * recall
436     /
437     (precision + recall + 1e-8)
438 )
439
440
441 print()
442 print("Precision:", precision)

```

```

443 print("Recall :", recall)
444 print("F1 :", f1)
445
446
447 # =====
448 # 14. EXAMPLE PREDICTIONS
449 # =====
450
451 print()
452 print("Example predictions")
453 print("-----")
454
455 for i in range(
456     min(10, len(X_test))
457 ):
458
459     print(
460         f"Patient {i+1:2d} | "
461         f"Probability = "
462         f"{y_prob[i,0]:.3f} | "
463         f"Prediction = "
464         f"{y_pred[i,0]} | "
465         f"Actual = "
466         f"{int(y_test[i,0])}"
467     )

```

41 Reading the Program as Mathematics

One of the best ways to learn Deep Learning is to connect every important piece of code to an equation.

Python	Mathematics
$z1 = X @ W1 + b1$	$Z_1 = XW_1 + b_1$
$h1 = \text{relu}(z1)$	$H_1 = \text{ReLU}(Z_1)$
$z2 = h1 @ W2 + b2$	$Z_2 = H_1W_2 + b_2$
$h2 = \text{relu}(z2)$	$H_2 = \text{ReLU}(Z_2)$
$z3 = h2 @ W3 + b3$	$Z_3 = H_2W_3 + b_3$
$y_hat = \text{sigmoid}(z3)$	$\hat{y} = \sigma(Z_3)$
$\text{loss} = \text{binary_cross_entropy}(\dots)$	$L = -\frac{1}{N} \sum_i [\dots]$
$\text{gradients} = \text{backward}(\dots)$	$\nabla_{\theta} L$
$W -= \text{lr} * \text{dW}$	$W \leftarrow W - \eta \nabla_W L$

42 What Have We Actually Built?

We have built a complete neural network without a Deep Learning framework.

The network contains:

Linear Transformation + Nonlinearity + Linear Transformation + Nonlinearity + Linear Transformation + Sig

The learning algorithm contains:

Forward $\rightarrow$ Loss $\rightarrow$ Backpropagation $\rightarrow$ Gradient Descent

Nothing fundamentally mysterious is happening.

43 Why Is This Called Deep Learning?

A single logistic regression model can be written as:

$$\hat{y} = \sigma(XW + b).$$

Our model is:

$$\hat{y} = f_3(f_2(f_1(X))).$$

The difference is the learned intermediate representations.

$$X \rightarrow H_1 \rightarrow H_2 \rightarrow \hat{y}.$$

Thus:

Deep Learning = Learning a hierarchy of representations

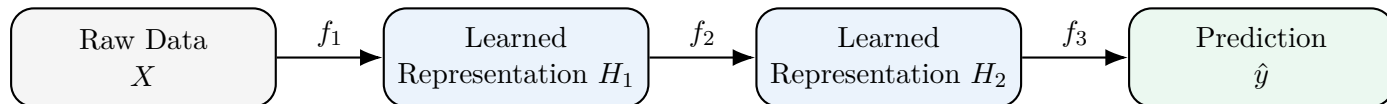
The first representation can capture relatively simple patterns.

The second representation combines these patterns.

The final layer maps the learned representation to the task output.

44 The Most Important Conceptual Picture

Students should remember the following picture:



And:

$$\hat{y} = f_3(f_2(f_1(X)))$$

This is the central equation of this tutorial.

45 From Scratch vs. PyTorch

Later in the course, students may see code such as:

```

1 model = nn.Sequential(
2     nn.Linear(8, 16),
3     nn.ReLU(),
4     nn.Linear(16, 8),
5     nn.ReLU(),
6     nn.Linear(8, 1),
7     nn.Sigmoid()
8 )
  
```

This may look very different from our NumPy implementation. But conceptually it is the same model.

From Scratch	Mathematics	Framework
$x @ W + b$	$XW + b$	Linear
$\text{relu}(x)$	$\text{ReLU}(x)$	ReLU
$\text{sigmoid}(x)$	$\sigma(x)$	Sigmoid
$\text{backward}()$	∇L	$\text{backward}()$
$W -= \text{lr} * dW$	$W - \eta \nabla W$	$\text{optimizer.step}()$

Important

A Deep Learning framework does not change the fundamental mathematics. It provides efficient and convenient implementations of the mathematics.

46 What Does PyTorch/TensorFlow Automate?

Our small program manually implements:

1. Parameter storage
2. Matrix multiplication
3. Activation functions
4. Loss calculation
5. Gradient calculation
6. Backpropagation
7. Parameter updates

Modern frameworks automate many of these operations. For example, automatic differentiation computes:

$$\frac{\partial L}{\partial W}.$$

Therefore, students should understand:

Framework $\neq$ Deep Learning itself

Rather:

Framework = Tools for implementing Deep Learning efficiently

47 A Small Numerical Example

Suppose one patient has a simplified representation:

$$H_2 = \begin{bmatrix} 1.2 & 0.5 & 0 & 2.0 & 0.4 & 0 & 1.1 & 0.3 \end{bmatrix}.$$

Suppose the final weights are approximately:

$$W_3 = \begin{bmatrix} 0.4 \\ -0.2 \\ 0.1 \\ 0.5 \\ 0.2 \\ -0.3 \\ 0.4 \\ 0.1 \end{bmatrix}$$

and

$$b_3 = 0.1.$$

Then:

$$Z_3 = H_2 W_3 + b_3.$$

Numerically:

$$\begin{aligned} Z_3 &= 1.2(0.4) + 0.5(-0.2) + 0(0.1) + 2.0(0.5) \\ &\quad + 0.4(0.2) + 0(-0.3) + 1.1(0.4) + 0.3(0.1) + 0.1. \end{aligned}$$

Therefore:

$$Z_3 \approx 1.53.$$

The sigmoid gives:

$$\hat{y} = \frac{1}{1 + e^{-1.53}} \approx 0.822.$$

Thus the model predicts:

$$P(\text{diabetes}) \approx 0.822$$

and, using a threshold of 0.5:

$$\hat{y}_{\text{class}} = 1.$$

48 Understanding the Role of Each Layer

A useful conceptual interpretation is:

Stage	Operation	Role
Input	Raw features	Describe patient
Layer 1	$8 \rightarrow 16$	Learn first representation
Layer 2	$16 \rightarrow 8$	Combine representations
Layer 3	$8 \rightarrow 1$	Produce prediction

Thus:

$$\text{Layer 1: learn}$$

$$\text{Layer 2: learn from learned features}$$

$$\text{Layer 3: predict}$$

49 Why Multiple Layers?

Suppose we used only:

$$8 \rightarrow 1.$$

Then the model would be approximately:

$$\hat{y} = \sigma(XW + b).$$

This is essentially logistic regression.

Now introduce:

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 1.$$

The network can construct nonlinear intermediate representations.

Therefore:

More layers $\Rightarrow$ more levels of learned transformation

However, more layers are not automatically better.

The architecture must match the problem.

50 A Critical Point: Deep Does Not Mean "Many Neurons"

Students often confuse:

- width
- depth

The number of neurons in a layer describes its **width**.

The number of sequential transformations describes **depth**.

For example:

$$8 \rightarrow 100 \rightarrow 1$$

is relatively wide but shallow.

While:

$$8 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 1$$

is deeper.

The central idea of Deep Learning is the composition:

$$f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1.$$

51 Exercises

Exercise

Exercise 1 – Run the Model

Download or obtain the diabetes dataset and save it as:

diabetes.csv

Run the complete program.

Record:

- Final training loss
- Test accuracy
- Precision
- Recall
- F1

Exercise

Exercise 2 – Change the Architecture

Change:

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

to:

$$8 \rightarrow 32 \rightarrow 16 \rightarrow 1.$$

Run the experiment again.
Compare the results.

Exercise

Exercise 3 – Change the Learning Rate

Run the model with:

$$\eta = 0.001$$

then:

$$\eta = 0.01$$

then:

$$\eta = 0.1.$$

Compare the loss curves.
What happens when the learning rate is too small?
What happens when it is too large?

Exercise

Exercise 4 – Remove Layer 2

Create a simpler network:

$$8 \rightarrow 16 \rightarrow 1.$$

Compare it with:

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 1.$$

Discuss whether the deeper model always performs better.

Exercise**Exercise 5 – Remove ReLU**

Modify the network so that the hidden layers do not use ReLU.

Compare the training behavior.

Explain why stacking linear transformations without nonlinearities does not provide the same expressive power as a nonlinear network.

Exercise**Exercise 6 – Change the Decision Threshold**

Instead of:

$$\hat{y} \geq 0.5,$$

try:

$$\hat{y} \geq 0.3$$

and

$$\hat{y} \geq 0.7.$$

Compare precision and recall.

Explain the trade-off.

Exercise**Exercise 7 – Inspect the Representations**

Modify the program to print:

$$H_1.shape$$

and

$$H_2.shape.$$

Explain why they are:

$$(N, 16)$$

and

$$(N, 8).$$

52 Challenge: Visualize the Training Process

Modify the program so that it stores the loss at every epoch:

```

1 loss_history = []
2
3 for epoch in range(epochs):
4
5     y_hat, cache = forward(X_train)
6
7     loss = binary_cross_entropy(
8         y_train,
```

```

9     y_hat
10 )
11
12 loss_history.append(loss)
13
14 gradients = backward(
15     y_train,
16     cache
17 )
18
19 W1 -= learning_rate * gradients["dW1"]
20 b1 -= learning_rate * gradients["db1"]
21
22 W2 -= learning_rate * gradients["dW2"]
23 b2 -= learning_rate * gradients["db2"]
24
25 W3 -= learning_rate * gradients["dW3"]
26 b3 -= learning_rate * gradients["db3"]

```

Then plot:

Epoch vs. Loss.

Students should observe that successful learning generally produces a decreasing training loss.

53 Challenge: Build Your Own Layer Function

Instead of writing each layer separately, create a general function:

```

1 def dense_layer(X, W, b):
2     return X @ W + b

```

Then:

```

1 z1 = dense_layer(X, W1, b1)
2 h1 = relu(z1)
3
4 z2 = dense_layer(h1, W2, b2)
5 h2 = relu(z2)
6
7 z3 = dense_layer(h2, W3, b3)
8 y_hat = sigmoid(z3)

```

This helps students recognize that a neural network is built by composing reusable functions.

54 Important Questions for Self-Study

Students should be able to answer these questions without looking at the code.

1. What is the difference between a traditional ML model and a Deep Learning model?
2. What does the equation

$$\hat{y} = f_3(f_2(f_1(X)))$$

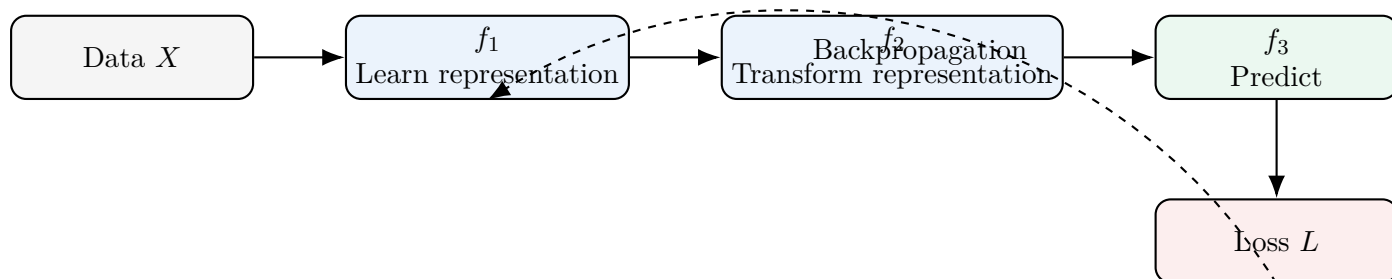
mean?

3. What is the purpose of W ?

4. What is the purpose of b ?
5. Why do we need nonlinear activation functions?
6. What does ReLU do?
7. Why do we use sigmoid in the final layer?
8. Why do we need a loss function?
9. What does binary cross-entropy measure?
10. What is a gradient?
11. What does backpropagation calculate?
12. What does gradient descent do?
13. Why are the weights updated repeatedly?
14. Why do we separate training and testing data?
15. Why do we normalize the features?
16. What is the difference between a probability and a class prediction?

55 The Complete Mental Model

Students should finish this tutorial with the following mental model.



The whole process is:

$$X \rightarrow f_1 \rightarrow f_2 \rightarrow f_3 \rightarrow \hat{y} \rightarrow L \rightarrow \nabla L \rightarrow \text{Update}$$

Then repeat.

56 The Big Picture

We can summarize the entire tutorial with four equations.

1. Representation

$$H_1 = \text{ReLU}(XW_1 + b_1)$$

2. Deeper Representation

$$H_2 = \text{ReLU}(H_1 W_2 + b_2)$$

3. Prediction

$$\hat{y} = \sigma(H_2 W_3 + b_3)$$

4. Learning

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Together:

$$\text{Deep Learning} = \text{Function Composition} + \text{Representation Learning} + \text{Optimization}$$

57 From This Tutorial to Modern Deep Learning

This small diabetes model is intentionally simple.

Real Deep Learning systems may contain:

- Hundreds or thousands of layers
- Millions or billions of parameters
- Convolutional layers
- Recurrent layers
- Attention mechanisms
- Transformers
- Residual connections
- Normalization layers
- GPUs and distributed computation

However, the fundamental pattern remains:

$$\text{Compose functions} \rightarrow \text{calculate loss} \rightarrow \text{differentiate} \rightarrow \text{update parameters}$$

The complexity of modern systems comes primarily from the architecture, scale, data, and optimization methods—not from abandoning these fundamental principles.

58 From NumPy to a Deep Learning Framework

After understanding this tutorial, students are ready to learn a framework.

The conceptual transition is:

$$\text{Implement manually} \rightarrow \text{understand} \rightarrow \text{automate}$$

A framework will provide convenient abstractions for:

- Tensors
- Layers
- Models
- Automatic differentiation
- Optimizers
- GPU computation
- Dataset management
- Training utilities

But the underlying idea remains:

$$\hat{y} = f(X; \theta)$$

and:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L.$$

59 Final Takeaway

Key Concept

Deep Learning is learning a composition of functions.

For our diabetes example:

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

corresponds to:

$$X \xrightarrow{f_1} H_1 \xrightarrow{f_2} H_2 \xrightarrow{f_3} \hat{y}$$

The model learns the parameters:

$$\theta = \{W_1, b_1, W_2, b_2, W_3, b_3\}$$

by repeatedly performing:

$$\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \text{Gradient Descent}$$

This is the foundation on which modern Deep Learning is built.

Intelligent System Development

From Data $\rightarrow$ Representation $\rightarrow$ Function $\rightarrow$ Intelligence